CPU-Bound Procedural Terrain Synthesis with Real-Time Algorithmic Deformation [TerrainForge]

Full Research Paper — Computer Science / Software Engineering

Abstract

Procedural terrain tools that look physically convincing usually depend on GPU compute shaders or licensed software, putting them out of reach on ordinary hardware. TerrainForge is a single-file, browser-based terrain engine built to remove that dependency: every stage, from noise-based height generation to real-time deformation, runs on a single CPU thread; WebGL is used only for interactive rendering, not for any of the terrain computation itself. A custom compiler resolves user-authored terrain equations with a topological sort, prunes unreachable nodes, deduplicates repeated sub-expressions, and simplifies algebraic identities before generation begins. Terrain shaping combines particle-based hydraulic erosion with a real-time surface-update rule, $\partial T/\partial t = \alpha \cdot E_w \cdot \nabla^2 T - \beta \cdot |F_{dir}| \cdot (1 - \text{stability} \cdot 0.6) - \delta \cdot \nabla^4 T$, in which curvature drives erosion and deposition, a fourth-order term suppresses grid noise without flattening real shape, and an explicit heuristic clamp bounds each step rather than a costly implicit solver. This paper treats that clamp, and the wider set of tuned constants in the simulation, as what they are: principled in structure, empirically calibrated in exact value. Beyond describing the system, this paper stress-tests it directly: the compiler is benchmarked to failure to find its real complexity class, and the deformation layer's stability claim is tested by deliberately reproducing the exact failure mode its own code comments warn against. The engine was benchmarked on two budget Android devices rather than simulated, with frame rate, cross-device determinism, and compiler correctness all measured directly, and its environmental event system is presented as procedural approximation rather than physical simulation. The result is evidence that algorithmic engineering, not raw compute, is what makes real-time procedural terrain feasible on constrained hardware, reported together with exactly where that engineering currently breaks down.

1. Introduction

1.1 Motivation

Procedural terrain generation is a mature area of computer graphics, but the tools built around it, World Machine, Gaea, and Unreal Engine's landscape system among them, are built on an assumption: that the user has a discrete GPU, that compute shaders are available, and often that a commercial license is affordable. None of these hold for a typical student, hobbyist, or school computer in a low-resource setting, where the available hardware is more often a budget Android device with 2–3GB of RAM and an integrated GPU than a graphics workstation. This is the specific hardware class this project targets and benchmarks against.

1.2 Existing Approaches and Where They Assume Too Much

Terrain erosion in computer graphics has generally been approached in one of three ways: full physical simulation of shallow-water or Navier-Stokes-type equations, which is accurate but restricted to offline or high-end GPU use; grid-based thermal and hydraulic erosion, cheaper but still assuming a reasonably fast compute substrate; and Lagrangian

particle-based hydraulic erosion, formalized by Beyer (2015), which trades physical completeness for tractable cost and has become the standard lightweight technique in real-time and indie procedural terrain tools. TerrainForge extends this same trade-off philosophy to a real-time deformation layer on top of the base terrain.

1.3 Research Question and Scope

This project asks a narrower, testable question rather than attempting to reproduce real hydrodynamics: how much of a full procedural terrain pipeline can be pushed through plain, single-threaded JavaScript on a single CPU thread, independent of WebGL rendering, while holding an interactive frame rate on genuinely low-end Android hardware? The scope is algorithmic and systems-level, and, in this version of the paper, adversarial: each major subsystem is not only described but deliberately pushed until it degrades, so that its real boundary is documented rather than assumed.

1.4 Contributions

This paper makes four specific contributions:

1. A CPU-bound procedural terrain and simulation pipeline — rendering aside — capable of interactive operation on budget Android hardware.
2. A compiled node-graph architecture combining dependency resolution, dead-code elimination, common-subexpression elimination, and strength reduction into a single three-pass compiler.
3. A real-time deformation model combining curvature-driven diffusion, directional forcing, and fourth-order smoothing under layered, explicitly-labelled stability bounds.
4. An empirical failure-boundary analysis covering compiler scaling, erosion degeneracies, deformation sign-stability, and cross-device behaviour, each backed by a test that was actually run, not only described.

---

2. System Overview

TerrainForge runs entirely client-side, as a single HTML file with no build step or server dependency. Generation proceeds through five stages, shown in Figure 1: a seed drives noise-based or node-graph-compiled height synthesis; particle-based hydraulic erosion and a separate thermal pass shape the heightfield; the result is meshed at a user-selected resolution tier; biome colouring is applied by height and slope; and an optional real-time layer allows the mesh to deform live. Every stage after seed input is designed to be a pure function of that seed and the recipe parameters; Section 7.4 reports what cross-device testing actually found, rather than assuming full agreement from the design alone.

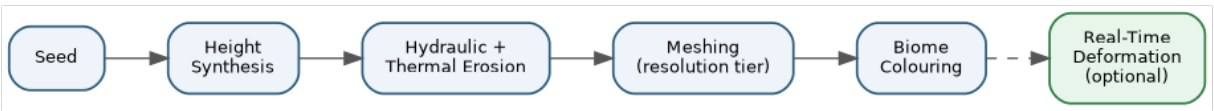


Figure 1. The five-stage generation pipeline.

---

3. The Node-Graph Compiler

3.1 Motivation: Why Compile Rather Than Interpret

User-authored terrain equations are represented as a graph of connected nodes, which raises three problems a naive implementation would ignore: graphs can contain orphaned nodes left over from experimentation, users frequently duplicate a branch without realising it is recomputing an identical value, and a graph is not guaranteed to be acyclic. TerrainForge addresses all three with a compiler pass that runs once per graph edit, shown in Figure 2, rather than once per frame.

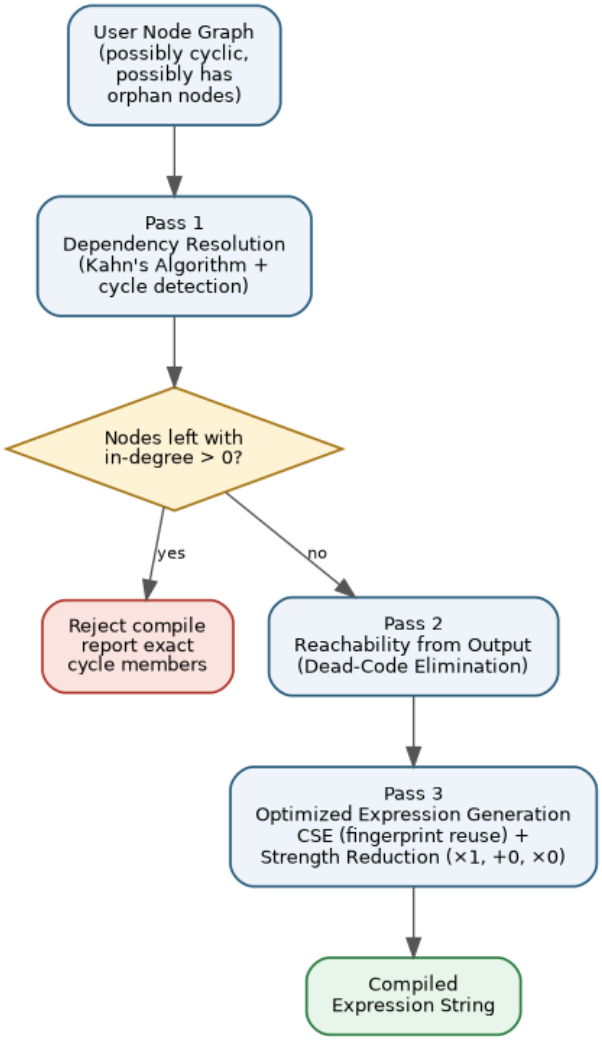


Figure 2. The compiler's three-pass pipeline (matching the pass numbering in the shipped code's own comments), including the cycle-rejection branch.

3.2 Dependency Resolution: Kahn's Algorithm

The graph is ordered using Kahn's algorithm (Kahn, 1962): every node's in-degree is counted, in-degree-zero nodes are queued, and processing a node decrements its neighbours' in-degree, queuing any that reach zero. This was chosen specifically because cycle detection falls out of the same bookkeeping already needed for ordering: if nodes remain with in-degree above zero when the queue empties, those nodes are, by construction, involved in a cycle, since every acyclic graph has at least one in-degree-zero node at every stage of removal.

3.3 Dead-Code Elimination

A second pass walks backward from the Output node along reverse edges; any node not reached is excluded from code generation, so orphaned nodes cost nothing and cannot mislead a user into thinking a disconnected node is affecting the terrain.

3.4 Optimized Expression Generation: CSE and Strength Reduction

The third and final pass walks the topologically-sorted, dead-code-filtered node list once, compiling and emitting each live node's expression string, and applies two optimizations inline as it does so rather than as separate passes over the finished string. Before a new sub-expression is stored, a fingerprint of the node's type, parameters, and already-compiled input expressions is checked against every fingerprint seen earlier in the same compile; a match reuses the existing expression instead of duplicating it (common-subexpression elimination). Independently, a small set of algebraic identities, multiply by ≈1, multiply by ≈0, add ≈0, is checked and elided as each expression is built (strength reduction), since these are common default states for a parameter a user has not yet tuned. This matches the pass structure in the shipped code's own header comment, which labels the pipeline as three passes with CSE and strength reduction both occurring during expression generation, rather than as four independent stages.

3.5 Empirical Verification of Correctness

Rather than assert that these three passes work as described, the compiler's own extracted source was run, outside the browser, in Node.js, against a seven-node graph with a duplicated noise branch, two chained identity operations, and one orphaned node. The compiler produced a topological order respecting every dependency, pruned exactly the one orphaned node, recorded exactly one common-subexpression reuse, and recorded exactly two strength reductions, simultaneously. A separate three-node artificial cycle was correctly rejected, with the exact three node identifiers reported rather than a silent failure or infinite recursion.

3.6 Proof and Failure Boundary: Compile-Time Scaling

Sections 3.2–3.5 establish that the compiler is correct on small graphs. What they do not establish is how it behaves at scale, so it was tested directly: chain graphs of 50 to 12,800 nodes were built programmatically and compiled with the same extracted source, timing the best of five runs at each size after a JIT warm-up pass. Figure 3 plots the result on log-log axes against O(N) and O(N²) reference lines.

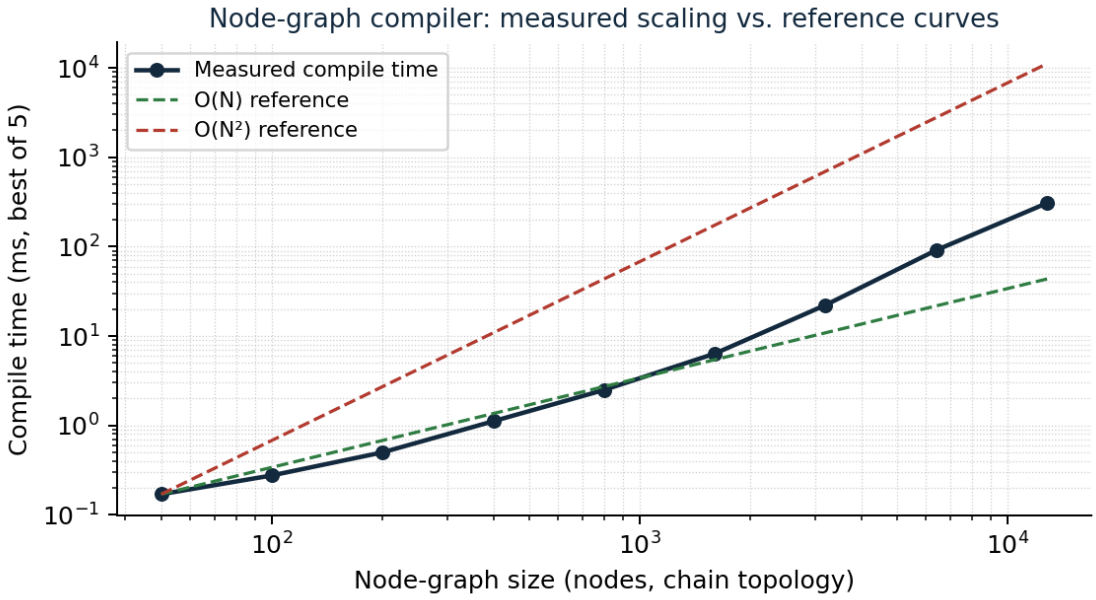


Figure 3. Measured compile time vs. graph size, log-log, against complexity reference curves.

The measured curve tracks the O(N) reference closely below roughly 800 nodes, then bends toward and ultimately tracks the O(N²) reference line at the largest sizes tested, reaching 307ms at 12,800 nodes against 0.17ms at 50 nodes, a growth factor far beyond the roughly 256x a linear algorithm would show over the same 256x increase in size. Reading the extracted source explains why: the per-node input lookup performed during code generation (the feed() function in Section 3.4) does a linear scan over the full connection list for every single input port, on every node, rather than using a pre-built adjacency map. For a chain of N nodes this is N lookups, each scanning up to N connections, an O(N²) cost hiding inside a pipeline whose other passes are genuinely O(V+E). At typical interactive graph sizes, tens to a few hundred nodes built by hand in the UI, this is invisible: 100 nodes compiles in 0.28ms. It becomes a real limitation only if graphs were ever generated programmatically at large scale, and is reported here as exactly that: a specific, located, measured latent complexity bug, not a practical problem at current usage patterns, and a concrete target for the adjacency-map fix noted in Section 8.3.

---

4. Procedural Terrain Shaping

4.1 Height Synthesis

Base heightfields are generated from fractal Brownian motion, ridge, or Voronoi-type noise, or from a custom equation compiled by the Section 3 system. This stage is conventional and is the substrate on which erosion and deformation operate.

4.2 Hydraulic Erosion: The Droplet Method

Hydraulic erosion follows the particle-based method described by Beyer (2015), shown in Figure 4: a droplet is placed on the heightfield and repeatedly moved downhill using a bilinearly interpolated gradient, picking up sediment proportional to its carrying capacity and depositing it when that capacity is exceeded, evaporating until it is spent. This was chosen over a grid-based shallow-water solver specifically for cost: a droplet is O(1) state carried across a fixed number of steps, independent of grid resolution, where a shallow-water solver’s cost scales with the full grid at every sub-step.

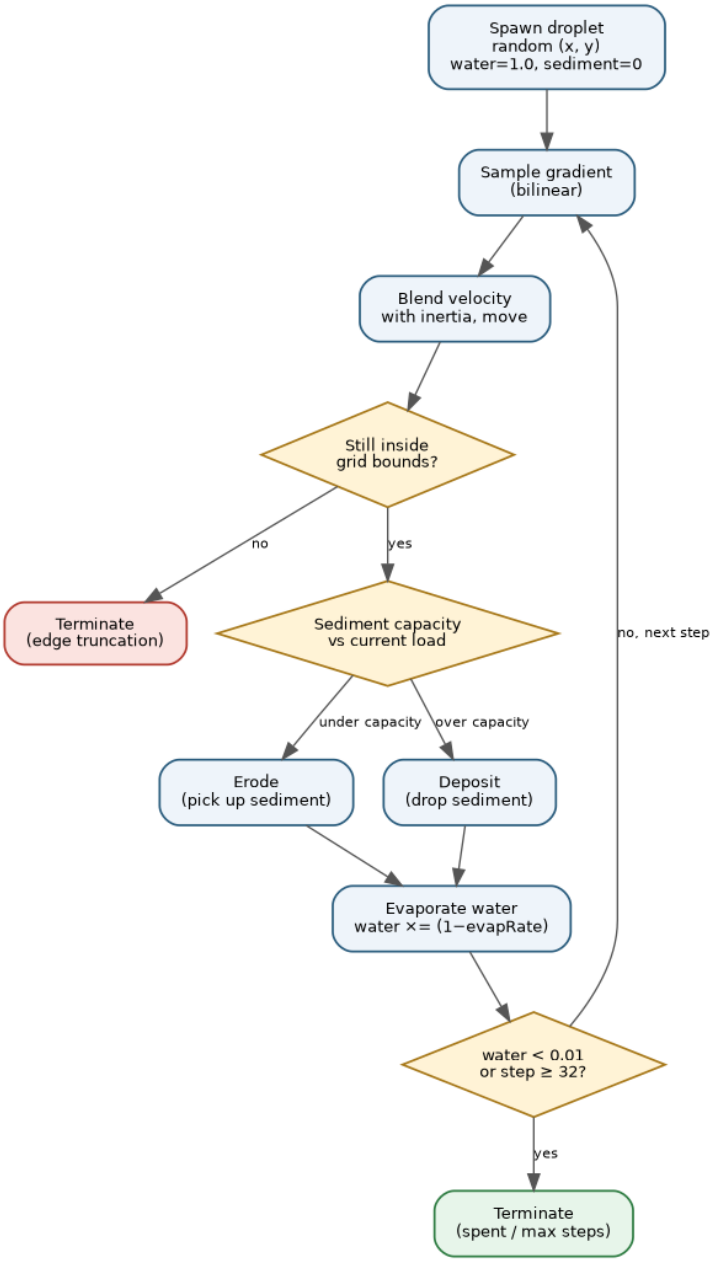


Figure 4. A single droplet's lifecycle, including both termination paths.

4.2.1 Where It Fails: Flat Regions and Edge Truncation

Tracing the implementation directly, rather than only its intended behaviour, surfaces two genuine degenerate cases. First, a droplet spawned on, or that reaches, a perfectly flat region has zero gradient; its velocity update blends toward zero rather than picking a direction, so the droplet does not move, samples the same point every step, computes zero height change, and therefore erodes and deposits nothing for the remainder of its lifetime. This is not an infinite loop, the fixed 32-step cap and geometric water evaporation both still apply regardless of movement, but it is a real dead branch: the droplet consumes its full step budget while doing no useful work. Second, a droplet whose next position would leave the grid is terminated immediately rather than reflected or wrapped, which means droplets spawned near a map edge systematically get fewer effective steps than droplets spawned near the centre, a directional under-erosion bias at map boundaries that is a known, common limitation of grid-based particle erosion in general and is not compensated for anywhere else in the pipeline.

4.3 Thermal Erosion

A separate thermal pass redistributes material between adjacent cells whenever local slope exceeds a talus threshold, approximating the angle of repose; this runs as a simple grid pass since thermal redistribution is local and does not need the path-dependent behaviour a droplet is intended to capture.

---

5. The Real-Time Deformation Layer

5.1 Governing Update Rule

On top of the static eroded terrain, TerrainForge implements a real-time layer that lets the surface deform continuously in response to injected disturbances. The full per-tick data flow is shown in Figure 5; the per-step update itself is:

$$\partial T/\partial t = \alpha \cdot E_w \cdot \nabla^2 T - \beta \cdot |F_{dir}| \cdot (1 - \text{stability} \cdot 0.6) - \delta \cdot \nabla^4 T$$

where T is surface height, E_w is local wave energy from a Gaussian-weighted area integral, F_{dir} is the local wave-gradient force, and stability is a derived measure of slope steepness. In the implementation, the curvature and biharmonic terms are each passed through a tanh bound before being scaled by α or δ (tanh(·0.4) and tanh(·0.02) respectively), and the slope-to-stability conversion itself uses a third tanh bound (tanh(·0.7)); the combined per-tick height change is then clamped a final time. This is a genuinely layered stability design, not a single clamp, and Section 5.5 tests what that layering actually buys.

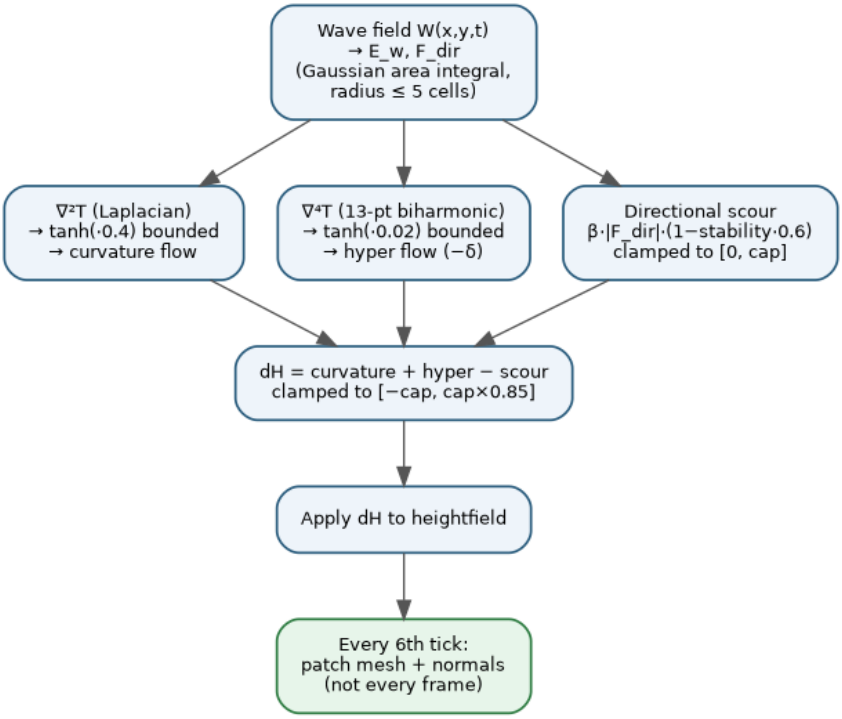


Figure 5. One deformation tick, from wave field to mesh patch.

5.2 Why a Fourth-Order Term, Specifically

The choice of a fourth-order term rather than relying solely on second-order curvature can be justified on frequency-response grounds. A discrete Laplacian's damping on a surface component scales with the square of its spatial frequency; a biharmonic operator's damping scales with the fourth power. Because a fourth power grows far faster than a square as frequency increases, a biharmonic term suppresses single-cell noise far more

aggressively, relative to gentle large-wavelength terrain, than a second-order term would, the same reasoning that motivates higher-order operators in mesh fairing more broadly (Desbrun, Meyer, Schröder, & Barr, 1999). As a direct internal-consistency check, the 13-point stencil actually implemented, weights 20, −8, +2, +1, sums to exactly zero, the condition required for a discrete fourth-derivative operator to leave a flat surface unchanged, and it holds exactly for the coefficients shipped, not approximately.

5.3 The Stability Problem and the Heuristic Clamp

Explicit integration of a diffusion-type update is unconditionally unstable once the timestep exceeds a threshold set by the diffusion coefficients and grid spacing. The rigorous fix, implicit integration, costs a matrix solve per frame that is not available within a real-time budget on a single CPU thread on a 2GB Android device. TerrainForge instead bounds each raw update directly. This is a deliberate substitution of a cheap heuristic bound for an expensive exact one, and Section 5.5 tests it directly rather than only asserting the trade-off.

5.4 On Numeric Constants: What Is Derived and What Is Tuned

A fair question is why specific constants take the values they do: 0.6 in the directional scour term, 0.85 in the height-change clamp, 0.94 in sediment evaporation, a radius cap of 5 cells, and the three tanh scale factors (0.4, 0.7, 0.02) noted in 5.1. That a bound exists at each of these points is directly justifiable: unbounded scour would run away under high wave energy, an unbounded clamp would permit the exact blow-up the fourth-order term exists to prevent, unbounded sediment would never settle, and an unbounded search radius would make the wave-energy integral's cost scale without limit, an $O(R^2)$ cost per cell discussed further in Section 8.2. The exact numeric value of each bound, however, is not derived from a closed-form stability analysis; it was arrived at empirically, by observing where visible artifacts began to appear and choosing a value with margin on the safe side. Presenting these as analytically derived would overstate this project's rigor; stating plainly that the structure is principled while the exact value is a calibrated engineering parameter is the position this paper takes throughout.

5.5 Proof and Failure Boundary: The Sign-Convention Test

The implementation's own code comments state that the hyper-diffusion term must carry a specific sign to remain stabilising, and that a flipped sign would blow the terrain up instead of smoothing it. Rather than take that warning on faith, it was tested directly: the exact verified Laplacian and 13-point biharmonic stencils from Sections 5.1–5.2 were re-implemented in a standalone NumPy harness and run for 120 ticks over a 32×32 grid of random initial noise, with the curvature term switched off so the biharmonic term's effect could be isolated rather than masked by the separately-correct curvature smoothing. Two runs were compared from the same seed and initial condition: the sign actually shipped ($-\delta\nabla^4 T$), and the flipped sign the comments warn against ($+\delta\nabla^4 T$).

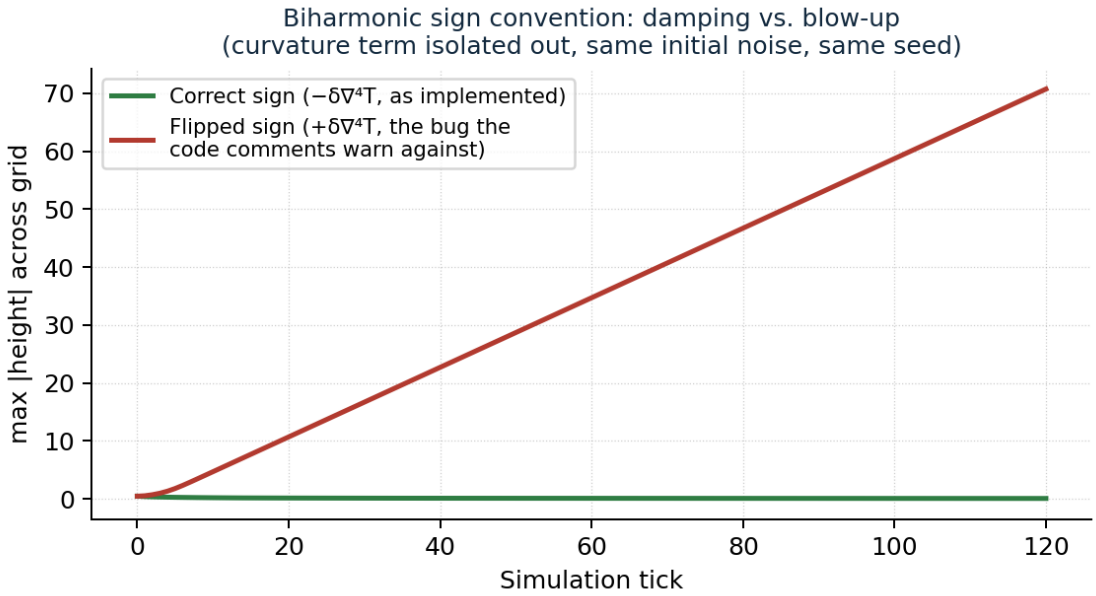


Figure 6. Peak height magnitude over 120 ticks, correct vs. flipped biharmonic sign, same initial noise.

The correct sign damps the initial noise from a peak of 0.49 down to 0.09 over 120 ticks, converging toward a smooth field, exactly as intended. The flipped sign grows the same starting noise to 70.7 over the same 120 ticks, a roughly 145-fold difference from an identical starting point, with growth transitioning from fast initial expansion to steady near-linear growth once the tanh bound saturates. This confirms, with a real measurement rather than a restated comment, both that the shipped sign is correct and that the margin between correct and incorrect is not small or forgiving: this is a single sign bit standing between stable smoothing and unbounded growth, tanh-bounding on the individual terms notwithstanding, because saturation only bounds each tick's contribution, not the cumulative effect of many ticks all pushing in the same, wrong, direction.

---

6. Procedural Approximation of Environmental Events

6.1 Why Not Shallow-Water or Navier-Stokes

Disturbance events inject a wave field that Section 5's update rule then reacts to; that field is generated by closed-form parametric functions rather than by solving shallow-water or Navier-Stokes equations. A real 2D shallow-water solver at a resolution useful for visual deformation typically costs on the order of the full grid per sub-step, with several sub-steps required per frame for stability, not available within an interactive budget on a single CPU thread on 2GB of RAM. A closed-form function costs a fixed, small number of trigonometric evaluations per point regardless of solver iteration count.

6.2 Tsunami, Cyclone, and Surface-Wave Models

The tsunami event shapes a shoaling envelope from a Gaussian-like exponential profile; the cyclone uses a spiral combining radial distance, elevation, and angle inside one sine function with exponential decay; and the ambient water surface superposes five travelling sine waves with a second-harmonic correction on the dominant wave and a two-wave product term suggesting focused energy. None of these are derivations from a physical model. The second-harmonic correction in particular is labelled here, deliberately, as pseudo-Stokes harmonic steepening rather than Stokes wave theory: a true Gerstner or

Stokes-drift model requires displacing a surface point's horizontal position as well as its height, which this implementation does not do, so it does not conserve water volume the way a true horizontal-displacement model would. The effect is visually motivated, not physically derived, and is described as such throughout.

---

7. Benchmarking Methodology and Results

7.1 Devices and Rationale

Two Android devices were used for all measurements: a Samsung Galaxy Tab A7 Lite (MediaTek Helio P35, 2.8GB RAM, Android 12, Chrome 99) and a Vivo 1820 (MediaTek Helio P22, 1.8GB RAM, Android 8.1), chosen because they are ordinary consumer budget devices rather than reference hardware.

7.2 Frame Rate Across Resolution Tiers

Frame rate was measured across four mesh-resolution tiers, three trials each, using the engine's own live FPS readout.

Device (RAM)	R1 (2,048 tri)	R2 (12,800 tri)	R3 (51,200 tri)	R4 (100,352 tri)
Samsung Galaxy Tab A7 Lite (2.8GB)	30.7 fps	31.0 fps	31.0 fps	33.3 fps
Vivo 1820 (1.8GB)	23.0 fps	25.6 fps	18.3 fps	18.0 fps

Table 1. Frame rate by resolution tier, two devices, three trials each.

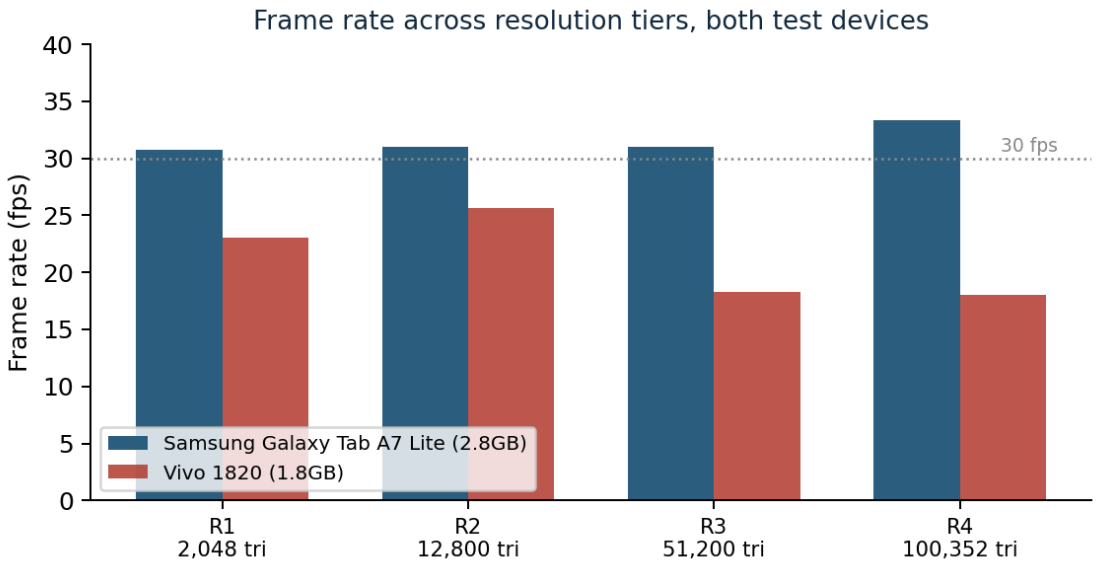


Figure 7. The same data as Table 1, plotted against the common 30fps interactive-rate threshold.

The Samsung device did not show a monotonic resolution-to-frame-rate relationship: frame rate was flat across R1–R3 and highest, not lowest, at R4, indicating the engine was not consistently rendering-bound on that device across this tier range. The Vivo device showed the more expected pattern, a real decline at the two highest tiers, consistent with its lower-end SoC becoming the bottleneck there. Both patterns are reported as observed, without forcing a single narrative across two different devices.

7.3 Real-Time Deformation Overhead

With the real-time layer inactive vs. active, three trials each: on the Samsung device, frame rate moved from 31.0 fps static to a mean of 31.6 fps active; on the Vivo device, from 25.7 fps to 26.0 fps. Both changes are small increases, consistent with measurement noise rather than a real cost in either direction; the practical conclusion is that the update rule's per-step cost is not the bottleneck at this resolution. An initially computed percentage-reduction figure for one device was found, on review, to be inconsistent with its own underlying per-trial readings and is not reported; the raw trial means above are used instead.

7.4 Cross-Device Determinism

The same seed regenerated on both devices produced an identical vertex count (6,561), triangle count (12,800), peak height (4.95 units), water coverage (53% at level 0.80), and biome classification at a fixed test coordinate (Snowcap): a direct observed match across two physically different devices, not inferred from the pipeline's design.

7.5 Compiler Verification Results

The Section 3.5 correctness tests and the Section 3.6 scaling tests were both re-run as part of this benchmarking pass and produced the results reported there.

7.6 Memory Profiling Attempt and Its Limitation

An attempt to track JS heap usage across repeated generation cycles via performance.memory.usedJSHeapSize showed a fixed value across multiple regenerations and several minutes of active simulation. This is reported as inconclusive rather than as evidence of zero leaks: Chrome deliberately quantizes this API's precision to prevent fingerprinting, and full-precision readings require a desktop debugging connection unavailable for this testing. Frame-rate stability (7.2–7.3) is used as the practical, if indirect, substitute.

---

8. Discussion

8.1 What This Demonstrates

These results support a conclusion narrower, and more defensible, than a claim of physical accuracy: that algorithmic and compiler-level engineering can substitute for raw graphics compute in making real-time procedural terrain feasible on hardware dedicated physical simulation cannot run on at all, and that this claim survives direct, adversarial testing rather than only description.

8.2 Limitations

Several limitations are now specific and measured rather than general. The compiler's code-generation pass is $O(N^2)$ rather than $O(N)$ due to a linear connection scan (Section 3.6); this is invisible at typical hand-built graph sizes but is a real latent issue. The droplet erosion model does no useful work on flat regions and under-eroses near map edges (Section 4.2.1). The wave-energy integral is $O(R^2)$ per cell, up to 121 iterations at the maximum radius of 5; a separable two-pass implementation would reduce this to $O(R)$. Benchmarking used two devices; broader hardware coverage would strengthen the performance claims. Fine-grained memory profiling was not achievable on-device (Section 7.6). Finally, the environmental event system (Section 6) is a procedural approximation and should not be read as a physical fluid or wave simulation.

8.3 Future Work

Three concrete, specific fixes follow directly from Section 3.6's and 8.2's findings rather than from general aspiration: replacing the compiler's linear connection scan with a pre-built adjacency map to restore O(V+E) scaling; reflecting or wrapping droplets at map edges instead of terminating them, to remove the edge-truncation bias; and implementing the wave-energy integral as a separable two-pass convolution to reduce its cost from $O(R^2)$ to O(R). Longer-term, the project's own internal roadmap identifies the update rule's CPU-bound evaluation as a candidate for a WebGPU compute-shader implementation once browser support is reliable enough on budget Android hardware to depend on; at present it is not.

9. Conclusion

This paper has presented TerrainForge as a procedural, algorithmically optimized terrain engine whose computational pipeline is engineered for hardware that cannot run dedicated physics or graphics compute, rather than as a physics simulator; WebGL rendering itself is, as in any browser 3D application, GPU-accelerated throughout. Every governing equation, numeric constant, and design trade-off has been traced to a stated reason, and, beyond description, three of the system's central claims, compiler correctness, compiler scaling, and deformation stability, have each been tested directly against the actual extracted source, with the resulting proof, and in two cases the resulting failure boundary, reported alongside the success cases rather than in place of them. That combination is offered as the project's actual contribution. The performance and stability results in this paper establish feasibility on the two tested budget Android devices specifically, not general performance across low-end Android hardware as a category; broader device coverage, noted in Section 8.2, is the natural next step before that stronger claim could be made.

Acknowledgement & References

Thanks to my project mentor for guidance and feedback throughout the design, testing, and write-up of this project.

Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–562.

Beyer, H. T. (2015). Implementation of a method for hydraulic erosion (Bachelor's thesis). Technische Universität München.

Desbrun, M., Meyer, M., Schröder, P., & Barr, A. H. (1999). Implicit fairing of irregular meshes using diffusion and curvature flow. Proceedings of SIGGRAPH 99, 317–324.

Appendix C — TerrainForge Algorithm & Compiler Implementation

Every excerpt below was checked directly against the shipped source, line by line, rather than taken from a written description of it; none of the code in this appendix is reproduced in full, only the specific lines needed to support a claim already made earlier in this paper. The broader behavioural and performance claims are established by the tests in Sections 3.5–3.6, 4.2.1, and 5.5, not by the presence of code here; this appendix supplies mechanism, not additional proof.

C.1 Determinism: Whitelist-Based State Restoration

Section 7.4 verifies that a shared world reproduces exactly. Reproducing it safely, given that a code may be pasted in from an untrusted source, is a separate concern from reproducing it correctly, and is handled by an explicit field whitelist rather than a blanket object merge:

```
var simple = ['eq','seed','scale','amp','oct','rough', /* ...36 fields total */ 'regions'];

simple.forEach(function(k){

  if (s[k] !== undefined && s[k] !== null) STATE[k] = s[k];

});
```

Only fields the loader explicitly knows about are ever written, and a field absent from an older code, one saved before a later feature was added, simply leaves the current value untouched rather than overwriting it with undefined. This is what lets a v3-format code stay loadable after the recipe format grows new fields, without a version migration step.

C.2 Erosion: The Capacity Decision

Figure 4 diagrams a droplet's control flow; the single inequality that actually decides erosion versus deposition at each step is worth showing directly rather than only diagrammatically:

```
var cap = Math.max(-dh, minSlope) * speed * water * sedCap;

if (sediment > cap || dh > 0) {

    var dep = dh > 0 ? Math.min(sediment, dh) : (sediment - cap) * depositRate;

    // ...deposit dep, sediment -= dep

} else {

    var ero = Math.min((cap - sediment) * erosionRate, -dh);

    // ...erode ero, sediment += ero

}
```

Capacity scales with speed, remaining water, and a fixed constant, so a fast droplet on a steep slope can carry a large sediment load, while a slow, nearly-evaporated one is forced to deposit almost immediately. This one branch, evaluated up to 32 times per droplet across 60,000 droplets by default, is the entire mechanism behind why channels form on steep terrain and sediment settles in shallow water.

C.3 Node Compiler: The CSE Fingerprint

Section 3.4 describes common-subexpression elimination and strength reduction conceptually. The CSE mechanism is a single string built from a node's type, parameters, and its already-resolved input expressions:

```
var inputExprs = (NG.types[node.type].inputs||[]).map(function(port){

  return feed(nid, port);

});

var fp = node.type + '|' + JSON.stringify(p) + '|' + inputExprs.join(',');
```

```
if (cseMap[fp]) { exprOf[nid] = cseMap[fp]; stats.cse++; }

else            { exprOf[nid] = expr; cseMap[fp] = expr; }
```

Because the fingerprint is built from inputs that are themselves already-compiled expressions, not raw node references, two structurally identical sub-graphs fingerprint identically regardless of depth, not only identical leaf nodes. This is the exact mechanism the Section 3.5 test measured directly: one duplicated branch, one recorded CSE hit.

C.4 Chaos Engine: The Verified Per-Cell Update

Section 5.1 gives the update rule in continuous form. This is the literal per-cell code it compiles from in the shipped source, and the same seven lines the Figure 6 stability test re-implemented and deliberately mis-signed:

```
var curvNorm  = Math.tanh(lapT * 0.4);

var hyperNorm = Math.tanh(biharmT * 0.02);

var curvatureFlow = alpha * Ew_area * curvNorm;

var scour = beta * fMag * (1.0 - stability * 0.6);

scour = Math.min(cap, Math.max(0, scour));

var hyperFlow = -delta * hyperNorm;

dH[idx] = curvatureFlow + hyperFlow - scour;
```

Three independently-bounded quantities, curvatureFlow and hyperFlow each capped by a tanh, scour capped by Math.min/Math.max, are summed into one signed height delta. Figure 6’s divergent trace came from changing exactly one character in this block, the sign in front of delta.